

Express.js · Requisições

Como enviar dados via requisição

Body · Query Params · Path Params

Três formas de enviar dados

Toda requisição pode carregar informações de três maneiras diferentes.

Cada uma tem seu lugar certo — usar a errada funciona, mas é como colocar o endereço do destinatário dentro do envelope em vez de escrever na frente.

01

Path Param

```
/usuarios/42
```

Parte da URL. Identifica qual recurso você quer.

02

Query Param

```
/produtos?pagina=2
```

Após o ?. Filtra, busca ou pagina resultados.

03

Body

```
{ "nome": "Lucas" }
```

Corpo da requisição. Dados de criação ou atualização.

O que é e como funciona

São valores embutidos diretamente no caminho da URL. Funcionam como o número de um apartamento — o endereço do prédio é a rota, o número é o path param.

```
GET /usuarios/42 ← o ":id" virou 42
```

```
// Definição da rota - o : indica que é um parâmetro dinâmico
app.get('/usuarios/:id', (req, res) => {
  const id = req.params.id // "42"
  res.json({ mensagem: `Buscando usuário ${id}` })
})
```

Você pode ter mais de um path param na mesma rota:

```
app.get('/usuarios/:id/pedidos/:pedidoId', (req, res) => {
  const { id, pedidoId } = req.params // id="42", pedidoId="8831"
})
```

Quando usar (e quando não usar)

Use quando...

- ✓ Quer identificar um recurso específico
- ✓ O dado faz parte da identidade da URL
- ✓ Está buscando, atualizando ou deletando um item

Não use quando...

- ✗ Quer filtrar ou buscar resultados
- ✗ Os dados são de um formulário ou criação
- ✗ O dado é opcional

Exemplos reais:

```
app.get('/usuarios/:id')           // buscar um usuário
app.put('/produtos/:id')           // atualizar um produto
app.delete('/pedidos/:id')         // deletar um pedido
app.get('/posts/:slug/comentarios') // comentários de um post
```

O que é e como funciona

Ficam após o ? na URL como pares chave=valor. São como os filtros do iFood — você está na mesma tela (rota), mas refinando o que quer ver.

```
GET /produtos?categoria=eletronicos &pagina=2
```

```
app.get('/produtos', (req, res) => {  
  const categoria = req.query.categoria // "eletronicos"  
  const pagina    = req.query.pagina    // "2" — sempre string!  
  const ordem     = req.query.ordem     // "preco"  
  
  res.json({ categoria, pagina, ordem })  
})
```

⚠️ Tudo que chega via query param é string — mesmo que pareça número. Converta com `Number()` quando precisar operar matematicamente.

Valores padrão e quando usar

Boa prática: sempre defina valores padrão. Query params são opcionais por natureza — o servidor não pode quebrar se o cliente não mandar.

```
app.get('/posts', (req, res) => {  
  // se não vier nada, usa o valor padrão  
  const pagina = Number(req.query.pagina) || 1  
  const limite = Number(req.query.limite) || 10  
  const ordem = req.query.ordem || 'recente'  
  
  // funciona com ou sem params na URL:  
  // GET /posts           → pagina=1, limite=10, ordem="recente"  
  // GET /posts?pagina=3   → pagina=3, limite=10, ordem="recente"  
})
```

Use para...

Filtros · Buscas · Paginação · Ordenação
Dados opcionais que não identificam o recurso

Nunca use para...

Dados sensíveis (senhas, tokens)
Query params ficam visíveis na URL e no histórico do navegador

O que é e como funciona

O corpo da requisição é o pacote de dados enviado junto — normalmente em JSON. É onde vivem dados complexos, formulários e informações sensíveis. Não aparece na URL.

Passo 1 — habilitar o middleware (uma vez só, no início):

```
app.use(express.json()) // sem isso, req.body chega undefined
```

Passo 2 — acessar os dados via req.body:

```
app.post('/usuarios', (req, res) => {  
  const { nome, email, senha } = req.body  
  
  // validação básica  
  if (!nome || !email || !senha) {  
    return res.status(400).json({ erro: 'Campos obrigatórios faltando' })  
  }  
  
  res.status(201).json({ mensagem: 'Usuário criado!', nome })  
})
```

⚠ app.use(express.json()) deve vir ANTES das rotas. Se vier depois, as rotas que precisam do body não vão encontrá-lo.

Body

Body em atualizações e quando usar

Em PUT e PATCH, é muito comum usar path param + body juntos: o path param diz qual recurso, o body diz o que mudar.

```
// PUT /usuarios/42
// Body enviado: { "nome": "Lucas Oliveira", "email": "novo@email.com" }

app.put('/usuarios/:id', (req, res) => {
  const { id } = req.params // path param: qual usuário
  const { nome, email } = req.body // body: o que atualizar

  res.json({ mensagem: `Usuário ${id} atualizado`, nome, email })
})
```

Use quando...

- ✓ POST — criar um recurso
- ✓ PUT/PATCH — atualizar um recurso
- ✓ Dados são muitos ou complexos
- ✓ Dados são sensíveis (senhas, tokens)

Não use quando...

- ✗ GET — GET não tem body por convenção
- ✗ Quer filtrar resultados (use query)
- ✗ Quer identificar um recurso (use path)

Na prática

Combinando os três juntos

Na vida real, os três aparecem juntos na mesma requisição. O segredo é saber o papel de cada um.

Exemplo 1 — listar pedidos de um usuário com filtro:

```
// GET /usuarios/42/pedidos?status=pendente&pagina=2
app.get('/usuarios/:id/pedidos', (req, res) => {
  const { id }      = req.params      // path:  "42"
  const { status } = req.query        // query: "pendente"
  const pagina      = Number(req.query.pagina) || 1 // query: 2

  res.json({ usuario: id, filtro: status, pagina })
})
```

Exemplo 2 — atualizar um produto específico:

```
// PUT /produtos/99   Body: { "preco": 1299.90, "estoque": 50 }
app.put('/produtos/:id', (req, res) => {
  const { id }      = req.params // path: "99"
  const { preco, estoque } = req.body // body: dados a atualizar

  res.json({ mensagem: `Produto ${id} atualizado`, preco, estoque })
})
```

Tabela comparativa

	Path Param	Query Param	Body	Acesso no Express
Onde fica	Na URL /user/42	Após o ? ?pg=2	Corpo JSON	<code>req.params</code> <code>req.query</code> <code>req.body</code>
Visível na URL	Sim	Sim	Não	—
Uso principal	Identificar um recurso	Filtrar / buscar / paginar	Criar ou atualizar dados	—
Métodos comuns	GET, PUT, DELETE	GET	POST, PUT, PATCH	—
Dados sensíveis	Não	Nunca	Sim	—

Para fixar

A regra de ouro

Path Param

QUAL recurso

/usuarios/42 → qual usuário?

Query Param

COMO você quer

?ordem=preco → de que forma?

Body

O QUE você manda

{ nome, email } → quais dados?

Se você sempre se fizer essas três perguntas antes de escrever a rota, vai acertar na maioria das vezes.

